

Refinamento Técnico — Guia Passo a Passo

Como construir o projeto Bikcraft: pastas, tags, código-modelo e ordem de execução

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semanas 07 a 10 | Este documento é o complemento técnico do **Documento Funcional — Projeto Bikcraft**. Enquanto aquele descreve **o que** cada página precisa ter, este mostra **como** construir: a organização de arquivos, os recursos de HTML/CSS/JS envolvidos, modelos de código para cada etapa e a ordem recomendada de construção.

1. Estrutura de pastas e arquivos

Crie uma única pasta de projeto com os arquivos abaixo. Todas as páginas compartilham o mesmo CSS e o mesmo JavaScript — evite criar um arquivo de estilo por página.

```
bikcraft/  
├─ index.html      // página inicial  
├─ bicicletas.html // catálogo "escolha a sua"  
├─ contato.html    // formulário + lojas  
├─ style.css       // uma única folha de estilos para as 3 páginas  
├─ script.js       // um único script para as 3 páginas  
└─ assets/  
    ├─ bike-1.svg (ou .png/.jpg)  
    ├─ bike-2.svg  
    └─ bike-3.svg
```

Por que um único CSS e um único JS?

Porque as 3 páginas compartilham cabeçalho, cores, fontes e alguns comportamentos. Um arquivo por página duplicaria código e criaria inconsistência visual entre elas — a mesma razão pela qual sites profissionais usam uma folha de estilos "global".

2. Como abrir e testar o projeto

- Abra a pasta `bikcraft/` inteira no VS Code (File → Open Folder).
- Instale a extensão **Live Server**, se ainda não tiver.
- Clique com o botão direito em `index.html` → **"Open with Live Server"** (ou clique em "Go Live" na barra inferior).
- Nunca abra as páginas com duplo clique direto no arquivo — os links entre páginas e alguns recursos podem não funcionar do mesmo jeito sem um servidor local.
- Para testar a responsividade: abra o DevTools (F12), ative o "modo dispositivo" (ícone de celular/tablet) e teste em 375px e em 1280px.

3. Tags e recursos que você vai usar

Tag / recurso	Para que serve
<code><meta name="viewport"></code>	Faz o celular usar a largura real da tela, em vez de simular um desktop
<code><header></code> , <code><nav></code> , <code><a></code>	Cabeçalho e menu de navegação entre páginas
<code><section></code> , <code><div></code> , <code><article></code>	Blocos de conteúdo (hero, colunas, cards de bicicleta/loja)
<code><h1></code> / <code><h2></code> / <code><h3></code>	Títulos e subtítulos, na ordem hierárquica correta
<code></code> + <code>max-width:100%; height:auto;</code>	Imagens que nunca ultrapassam a largura do container
<code><form></code> , <code><label for></code> , <code><input></code> , <code><textarea></code> , <code><button></code>	Formulário de contato, com rótulos ligados aos campos
<code>id</code> × <code>class</code>	<code>id</code> identifica UM elemento; <code>class</code> agrupa VÁRIOS que compartilham estilo/comportamento
<code>display: flex</code> , <code>flex-wrap</code> , <code>gap</code> , <code>flex</code> , <code>flex-direction</code>	Organizar itens em linha/coluna, com quebra e espaçamento controlados
<code>@media (min-width: ...)</code>	Aplicar regras de CSS só a partir de determinada largura de tela (mobile-first)
<code>clamp(mín, ideal, máx)</code>	Tamanho de fonte que se adapta à tela sem media query
Font Awesome — <code><i class="fa-solid ..."></code>	Ícones (telefone, e-mail, localização, redes sociais), via CDN
Bootstrap — <code>.btn</code> , <code>.card</code> , <code>.row</code> , <code>.col-md-6</code>	Componentes prontos, via CDN, customizados com uma classe própria por cima
Especificidade CSS	Decide qual regra "vence" quando duas miram o mesmo elemento — nunca resolve um conflito com <code>!important</code>
<code>getElementById</code> , <code>querySelectorAll</code>	Selecionar um elemento específico ou um grupo de elementos, em JavaScript
<code>addEventListener('submit', ...)</code> + <code>evento.preventDefault()</code>	Capturar o envio do formulário sem recarregar a página
<code>.value</code> , <code>.checked</code>	Ler o que o usuário digitou/marcou em cada campo
<code>classList.add/remove</code>	Ligar/desligar uma classe CSS via JavaScript (ex.: aviso de campo vazio)

4. Passo a passo de construção

Siga esta ordem — cada passo reaproveita o que já foi construído no anterior. Não pule etapas.

Passo 1 — Esqueleto HTML e cabeçalho compartilhado

O que fazer: criar os três arquivos HTML com a estrutura básica e o mesmo cabeçalho nos três.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bikcraft</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="cabecalho">
    <span class="logo">bikcraft</span>
    <nav>
      <a href="bicicletas.html">Bicicletas</a>
      <a href="#seguros">Seguros</a>
      <a href="contato.html">Contato</a>
    </nav>
  </header>

  <!-- o conteúdo de cada página entra aqui -->

</body>
</html>
```

Dica

Copie exatamente o mesmo `<header>` nos três arquivos — é assim que o menu funciona igual em todas as páginas.

Passo 2 — Home: o hero

O que fazer: dentro de `index.html`, montar a seção de apresentação com título, texto, botão e imagem.

```
<section class="hero">
  <div class="hero-texto">
    <h1 class="hero-titulo">Bicicletas feitas sob medida<span class="ponto-destaque">.</span></h1>
    <p class="hero-paragrafo">
      Bicicletas elétricas de alta precisão e qualidade, feitas sob medida para você.
    </p>
    <a class="botao-destaque" href="bicicletas.html">ESCOLHA A SUA</a>
  </div>
```

```
<div class="hero-imagem">
  
</div>
</section>
```

Atenção

O botão "ESCOLHA A SUA" é um link (`<a>`), não um `<button>` — porque a ação dele é navegar para outra página, e navegação é sempre trabalho do `<a>`.

Passo 3 — Catálogo: grade de bicicletas com id e class

O que fazer: em `bicicletas.html`, montar os cards de bicicleta com identificação individual e compartilhada.

```
<div class="grade-bicicletas">

  <article id="bike-1" class="card-bicicleta">
    
    <div class="info-card">
      <p class="nome-bike">Magic Might</p>
      <p class="preco-bike">R$ 2499</p>
    </div>
  </article>

  <!-- repita a estrutura para bike-2 e bike-3, mudando id, imagem, nome e preço -->

</div>
```

Por que id E class ao mesmo tempo?

`id="bike-1"` permite pegar SÓ a primeira bicicleta em JavaScript; `class="card-bicicleta"` permite pegar TODAS de uma vez com `querySelectorAll`. Os dois papéis são diferentes e os dois são exigidos no requisito da página.

Passo 4 — Script: mostrando os dados no console

O que fazer: em `script.js`, selecionar os elementos do catálogo e exibir as informações pedidas.

```
// Pega TODOS os cards que compartilham a classe.
const cardsBicicletas = document.querySelectorAll(".card-bicicleta");

// console.log 1 — nome e preço de TODAS as bicicletas.
cardsBicicletas.forEach((card) => {
  const nome = card.querySelector(".nome-bike").textContent;
  const preco = card.querySelector(".preco-bike").textContent;
  console.log(`${nome} - ${preco}`);
});

// A Bicicleta 1 tem um id próprio.
const bike1 = document.getElementById("bike-1");

// console.log 2, 3 e 4 — nome, preço e imagem SÓ da Bicicleta 1.
```

```
console.log(bike1.querySelector(".nome-bike").textContent);  
console.log(bike1.querySelector(".preco-bike").textContent);  
console.log(bike1.querySelector(".imagem-bike").src);
```

Não esqueça de importar o script no final do `<body>` de `bicicletas.html` : `<script src="script.js"></script>` .

Passo 5 — Contato: estrutura do formulário

O que fazer: em `contato.html`, montar o formulário com todos os campos exigidos, sem `required` (a validação é nossa, via JS).

```
<form id="form-contato" class="formulario-contato" novalidate>

  <div class="campo" id="grupo-nome">
    <label for="nome">Nome</label>
    <input type="text" id="nome" name="nome">
    <p class="mensagem-erro escondida" id="erro-nome">Preencha o nome.</p>
  </div>

  <!-- repita o padrão para telefone (type="tel"), email (type="email")
       e mensagem (<textarea>) -->

  <button type="submit">Enviar mensagem</button>
  <p class="mensagem-confirmacao escondida" id="mensagem-confirmacao"></p>
</form>
```

Passo 6 — Script: captura, validação e confirmação

O que fazer: escutar o envio do formulário, impedir o recarregamento, validar os campos e mostrar a confirmação.

```
const formContato = document.getElementById("form-contato");
const mensagemConfirmacao = document.getElementById("mensagem-confirmacao");

formContato.addEventListener("submit", (evento) => {
  evento.preventDefault(); // impede o recarregamento da página

  const nome = document.getElementById("nome");
  let valido = true;

  if (nome.value.trim() === "") {
    document.getElementById("grupo-nome").classList.add("campo-invalido");
    document.getElementById("erro-nome").classList.remove("escondida");
    valido = false;
  }

  // repita a verificação para telefone, e-mail e mensagem...

  if (!valido) return; // para aqui se algum campo estiver vazio

  const email = document.getElementById("email").value;
  mensagemConfirmacao.textContent =
    `Recebemos seu contato, ${nome.value}! Retornaremos no e-mail ${email}.`;
  mensagemConfirmacao.classList.remove("escondida");
});
```

Atenção ao return

O `return` depois do `if (!valido)` é o que impede a mensagem de sucesso de aparecer junto com o erro. Sem ele, as duas mensagens ficariam visíveis ao mesmo tempo.

Passo 7 — Responsividade: mobile-first

O que fazer: ajustar o cabeçalho e os títulos para funcionarem bem do celular ao desktop.

```
/* base = celular */
.cabecalho { padding: 16px 16px; }

.hero-titulo {
  font-size: clamp(2.25rem, 6vw, 3.75rem);
}

/* só a partir do desktop (~1024px) */
@media (min-width: 64em) {
  .cabecalho { padding: 16px 100px; }
}

img { max-width: 100%; height: auto; }
```

Teste arrastando a largura da janela do navegador (ou pelo DevTools) em 375px e em 1280px.

Passo 8 — Layout em 2 colunas com Flexbox

O que fazer: organizar a página de contato em duas colunas (dados e formulário), empilhadas no celular.

```
.secao-contato {
  display: flex;
  flex-direction: column; /* celular: empilhado */
  gap: 40px;
}

@media (min-width: 64em) {
  .secao-contato { flex-direction: row; gap: 56px; }
  .coluna-dados { flex: 5; }
  .coluna-formulario { flex: 7; }
}

/* nome e telefone na mesma linha, com quebra automática */
.linha-formulario {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.linha-formulario .campo { flex: 1 1 200px; }
```

Passo 9 — Bibliotecas prontas: Bootstrap e Font Awesome

O que fazer: incluir as bibliotecas via CDN e usá-las com moderação, sem perder a identidade visual da marca.

```
<!-- no <head>, DEPOIS do link do style.css NÃO — o Bootstrap vem ANTES: -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@4.7.0/css/font-awesome.min.css">
<link rel="stylesheet" href="style.css"><!-- o nosso vem por último -->
```

```
<!-- ícone -->
<i class="fa-solid fa-phone"></i>

<!-- botão do Bootstrap customizado com uma classe própria -->
<button type="submit" class="btn btn-bikcraft">Enviar mensagem</button>
```

```
/* style.css — vence o .btn do Bootstrap por vir depois no HTML,
   com a mesma especificidade (uma classe), sem !important */
.btn-bikcraft {
  background-color: #ffbf00;
  color: #1a1300;
  border: none;
}
```

Por que a ordem dos <link> importa?

Quando duas regras têm a mesma especificidade (uma classe contra uma classe), vence a que foi carregada por último. Por isso o `style.css` do projeto sempre entra depois do Bootstrap.

Passo 10 — Acabamento: transições e :hover

O que fazer: adicionar o toque final de suavidade nos cards e nos ícones do rodapé.

```
.card-loja {
  transition: transform 0.2s ease; /* fica no estado normal, não só no :hover */
}
.card-loja:hover {
  transform: scale(1.03);
}

.redes-sociais a {
  color: #f5f5f5;
  transition: color 0.2s ease;
}
.redes-sociais a:hover {
  color: #ffbf00;
}
```

5. Checklist de testes antes de entregar

- Abra as 3 páginas com o Live Server e navegue pelo menu — todos os links funcionam?
- Abra o DevTools (F12 → Console) em cada página — nenhum erro em vermelho?
- Modo dispositivo: teste em 375px e em 1280px — nada quebra, nada estoura a tela?
- No catálogo: os 4 `console.log` aparecem certos, na ordem certa?
- No contato: enviar o formulário vazio mostra o aviso vermelho? Enviar preenchido mostra a confirmação com o nome e o e-mail certos?
- O botão de envio está com a cor da marca (não a cor padrão azul do Bootstrap)?
- Alguma regra do seu CSS usa `!important` ? Se sim, revise a especificidade em vez disso.

Desenvolvimento Front-End [React] T4 — SENAI/SC — Projeto Bikcraft (Semanas 07 a 10)
Material de apoio ao aluno